

NATIONAL UNIVERSITY OF MODERN LANGUAGES
ISLAMABAD



Parallel and Distributed Computing (LAB)

Lab Report: 04

Submitted to
Sir Farhad Riaz

Submitted By
Junaid Asif
(BSAI-144)

Submission Date: October 20, 2025

Task 01

Problem Statement

Implement a parallel for loop using OpenMP to process iterations concurrently but display some outputs in an ordered manner, ensuring sequence preservation.

Code

```
#include <iostream>
#include <stdio.h>
#include <omp.h>

int main()
{
    int i;
    int sum = 0;

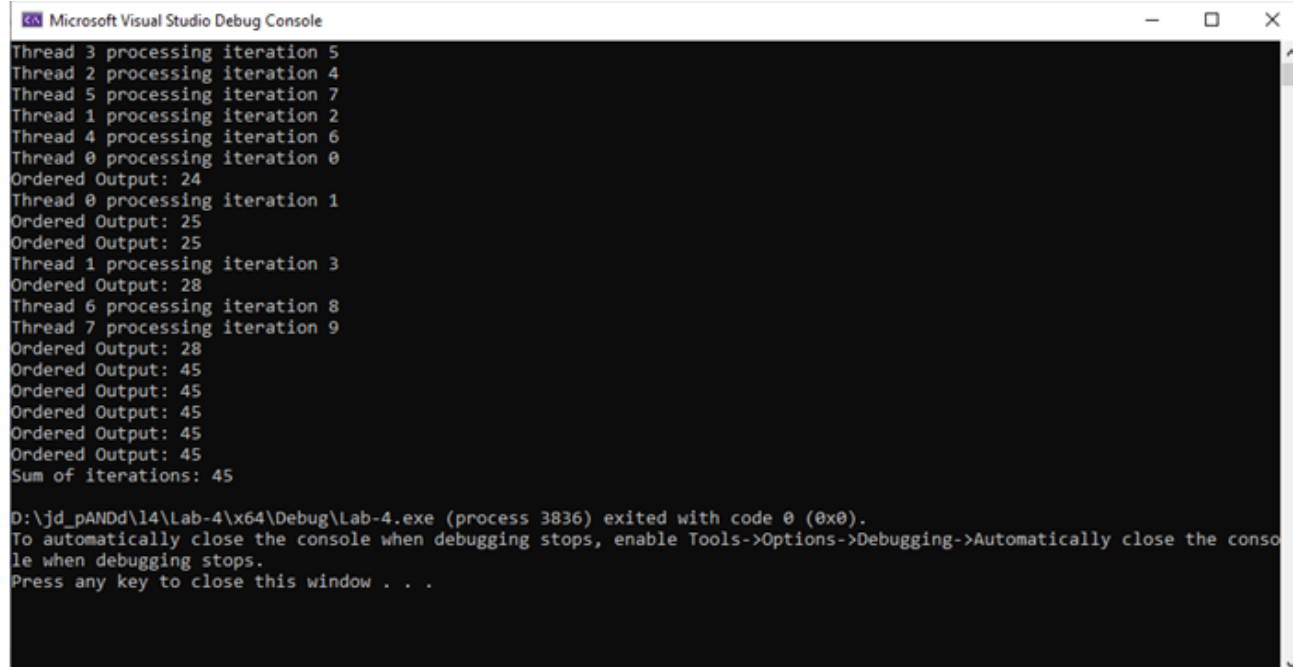
    #pragma omp parallel for ordered
    for (i = 0; i < 10; i++)
    {
        printf("Thread %d processing iteration %d\n", omp_get_thread_num(), i);
        sum += i;
    }

    #pragma omp ordered
    {
        printf("Ordered Output: %d\n", sum);
    }

    printf("Sum of iterations: %d\n", sum);

    return 0;
}
```

Output



```
Microsoft Visual Studio Debug Console
Thread 3 processing iteration 5
Thread 2 processing iteration 4
Thread 5 processing iteration 7
Thread 1 processing iteration 2
Thread 4 processing iteration 6
Thread 0 processing iteration 0
Ordered Output: 24
Thread 0 processing iteration 1
Ordered Output: 25
Ordered Output: 25
Thread 1 processing iteration 3
Ordered Output: 28
Thread 6 processing iteration 8
Thread 7 processing iteration 9
Ordered Output: 28
Ordered Output: 45
Ordered Output: 45
Ordered Output: 45
Ordered Output: 45
Ordered Output: 45
Sum of iterations: 45

D:\jd_pANDd\l4\Lab-4\Debug\Lab-4.exe (process 3836) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Explanation

- **What:** Demonstrates the use of the ordered clause to maintain sequential output inside a parallel for loop.
- **Why:** In parallel execution, outputs may appear out of order due to multiple threads writing simultaneously. The ordered clause ensures that a specific block executes in the original loop order.
- **How:**
 - `#pragma omp parallel for ordered` divides loop iterations among threads.
 - `#pragma omp ordered` ensures the code block inside executes in order, avoiding mixed or unpredictable console outputs.

Comparative Insight

Without ordered, the output becomes interleaved and difficult to interpret. The ordered clause adds synchronization overhead but provides readable, predictable results.

Task 02

Problem Statement

Demonstrate how the master directive restricts a code block to be executed by only the master thread.

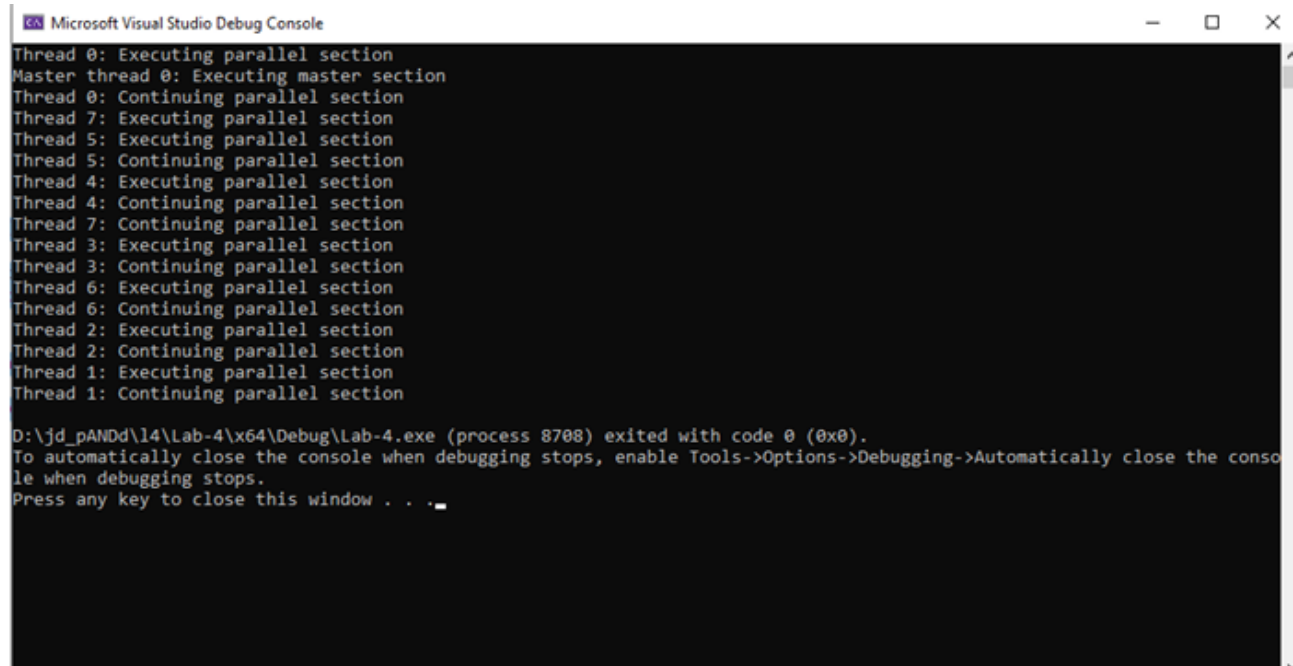
Code

```
#include <iostream>
#include <stdio.h>
#include <omp.h>
int main()
{
    int thread_id;
#pragma omp parallel private(thread_id)
    {
        thread_id = omp_get_thread_num();
        printf("Thread %d: Executing parallel section\n", thread_id);

#pragma omp master
        {
            printf("Master thread %d: Executing master section\n", thread_id);
        }

        printf("Thread %d: Continuing parallel section\n", thread_id);
    }
    return 0;
}
```

Output



```
Microsoft Visual Studio Debug Console
Thread 0: Executing parallel section
Master thread 0: Executing master section
Thread 0: Continuing parallel section
Thread 7: Executing parallel section
Thread 5: Executing parallel section
Thread 5: Continuing parallel section
Thread 4: Executing parallel section
Thread 4: Continuing parallel section
Thread 7: Continuing parallel section
Thread 3: Executing parallel section
Thread 3: Continuing parallel section
Thread 6: Executing parallel section
Thread 6: Continuing parallel section
Thread 2: Executing parallel section
Thread 2: Continuing parallel section
Thread 1: Executing parallel section
Thread 1: Continuing parallel section

0:\jdd_pANDd\l4\Lab-4\x64\Debug\Lab-4.exe (process 8708) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Explanation

- **What:** Illustrates the role of the master thread (thread 0) executing specific code while other threads skip it.
- **Why:** Sometimes initialization, I/O, or coordination tasks should only be handled by a single thread to prevent race conditions.
- **How:**
 - All threads enter the parallel region.
 - The `#pragma omp master` section runs **only** by thread 0.
 - Other threads continue after it, skipping the master block.

Comparative Insight

Compared to single, master does **not** have implicit barriers. It's faster but may lead to unsynchronized behavior if synchronization is required afterward.

Task 03

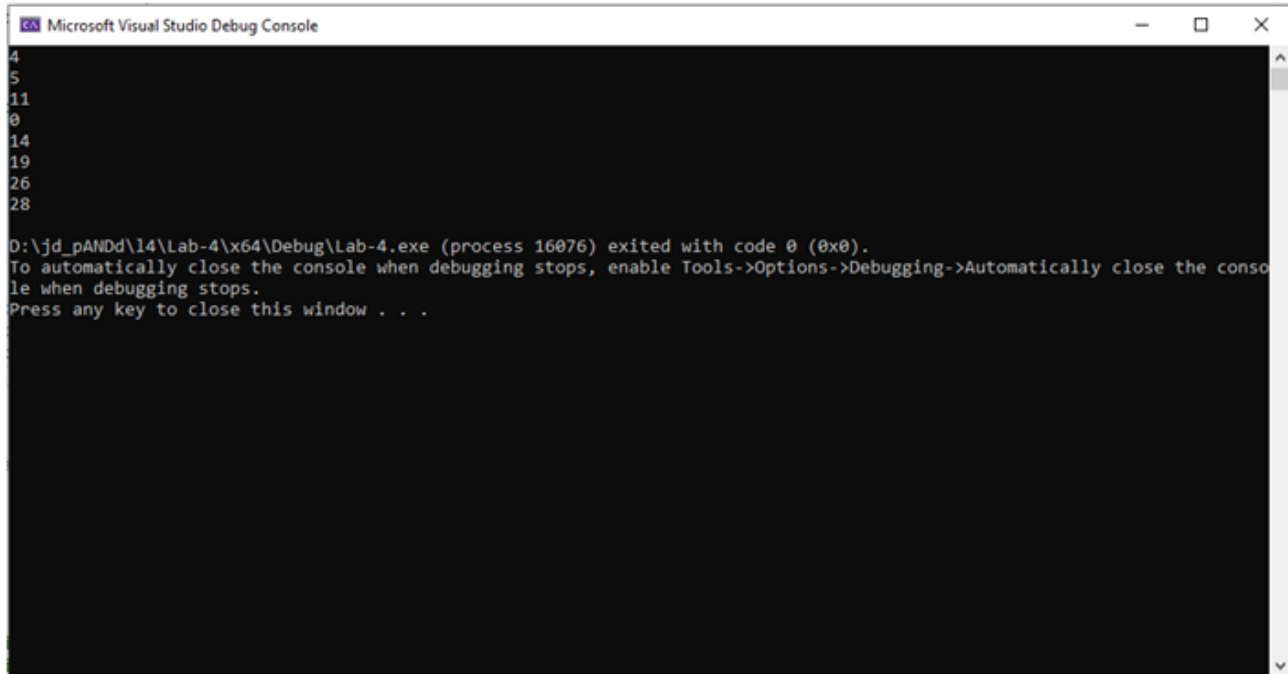
Problem Statement

Demonstrate how a shared variable behaves across multiple threads in OpenMP.

Code

```
#include <iostream>
#include <stdio.h>
#include <omp.h>
int main()
{
    int x = 0;
    #pragma omp parallel shared(x)
    {
        x += omp_get_thread_num();
    }
    printf("%d\n", x);
    return 0;
}
```

Output



```
4
5
11
0
14
19
26
28

D:\jd_pANDd\l4\Lab-4\x64\Debug\Lab-4.exe (process 16076) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Explanation

- **What:** Tests the impact of sharing a variable between threads.
- **Why:** Shared variables are accessible and modifiable by all threads — useful for global state but prone to **race conditions**.
- **How:**
 - `shared(x)` makes variable `x` accessible by all threads.
 - Each thread adds its ID to `x`, causing potential data inconsistency because of simultaneous writes.

Comparative Insight

`shared` provides accessibility but risks **data races** without synchronization (critical, atomic, or reduction).

Task 04

Problem Statement

Use OpenMP to calculate the sum of array elements in parallel with proper sharing and privatization of variables.

Code

```
#include <iostream>
```

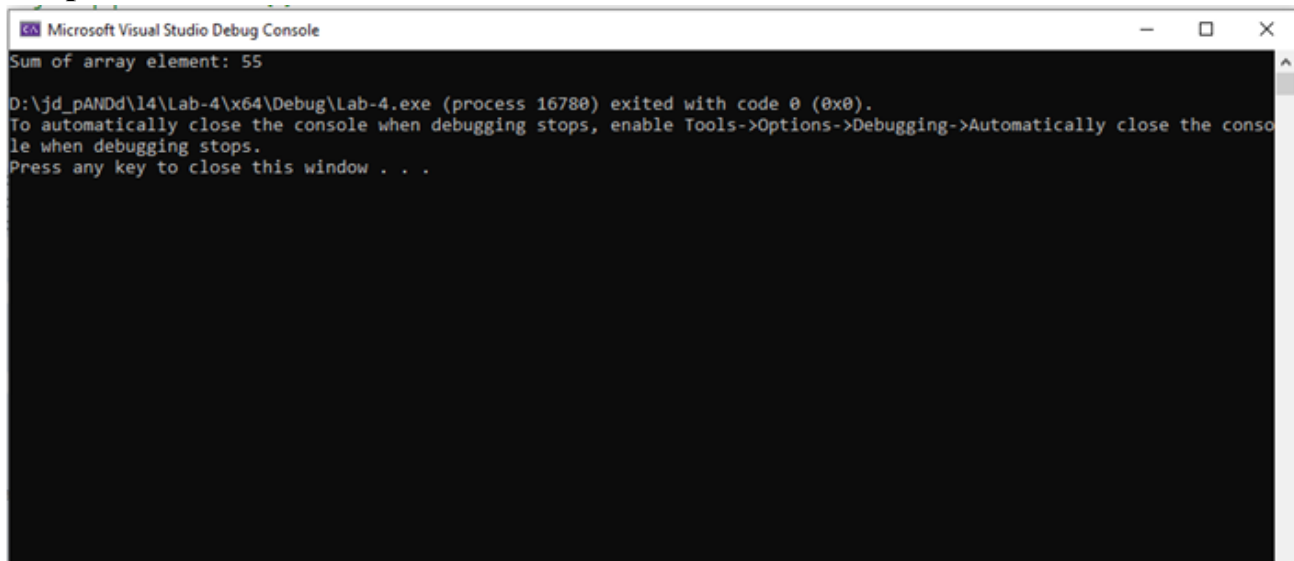
```
#include <stdio.h>
#include <omp.h>
int main()
{
    int i, n = 10;
    int sum = 0;
    int arr[200];

    for (i = 0; i < n; i++)
        arr[i] = i + 1;

    #pragma omp parallel for shared(sum, arr) private(i)
    for (i = 0; i < n; i++)
    {
        sum += arr[i];
    }

    printf("Sum of array elements: %d\n", sum);
    return 0;
}
```

Output

A screenshot of the Microsoft Visual Studio Debug Console window. The window title is "Microsoft Visual Studio Debug Console". The output text is: "Sum of array element: 55", "D:\jd_pANDd\l4\Lab-4\x64\Debug\Lab-4.exe (process 16780) exited with code 0 (0x0).", "To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.", and "Press any key to close this window . . .".

```
Microsoft Visual Studio Debug Console
Sum of array element: 55
D:\jd_pANDd\l4\Lab-4\x64\Debug\Lab-4.exe (process 16780) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Explanation

- **What:** Demonstrates parallel summation using shared and private variables.

- **Why:** To distribute workload among threads and reduce total computation time.
- **How:**
 - Each thread gets its own copy of `i` (private).
 - `sum` is shared, but since it's updated by all threads, race conditions may occur — ideally needs `reduction(+:sum)`.

Comparative Insight

This task shows how incorrect sharing can lead to inconsistent results. Compared to Task 03, this involves both shared and private control.

Task 05

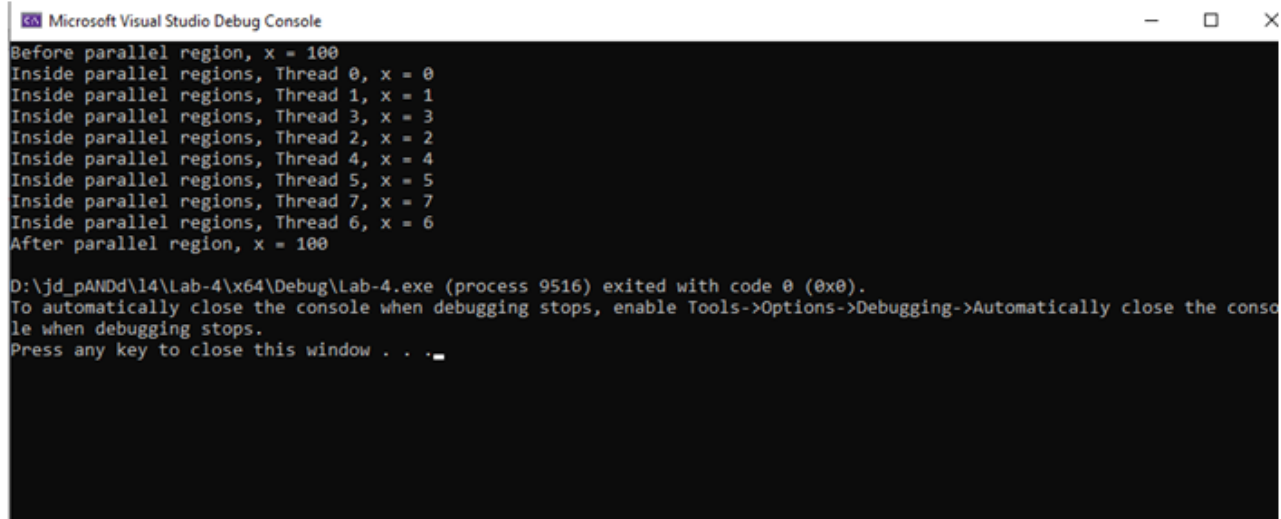
Problem Statement

Demonstrate the use of private variables in a parallel region and how their scope affects output.

Code

```
#include <iostream>
#include <stdio.h>
#include <omp.h>
int main()
{
    int x = 100;
    printf("Before parallel region, x = %d\n", x);
    #pragma omp parallel private(x)
    {
        int thread_id = omp_get_thread_num();
        x = thread_id;
        printf("Inside parallel region, Thread %d, x = %d\n", thread_id, x);
    }
    printf("After parallel region, x = %d\n", x);
    return 0;
}
```

Output



```
Microsoft Visual Studio Debug Console
Before parallel region, x = 100
Inside parallel regions, Thread 0, x = 0
Inside parallel regions, Thread 1, x = 1
Inside parallel regions, Thread 3, x = 3
Inside parallel regions, Thread 2, x = 2
Inside parallel regions, Thread 4, x = 4
Inside parallel regions, Thread 5, x = 5
Inside parallel regions, Thread 7, x = 7
Inside parallel regions, Thread 6, x = 6
After parallel region, x = 100

D:\jd_pANDd\l4\Lab-4\x64\Debug\Lab-4.exe (process 9516) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Explanation

- **What:** Tests how private creates individual copies of variables for each thread.
- **Why:** To prevent threads from interfering with each other's data.
- **How:**
 - Each thread has its **own copy** of x.
 - Changes inside the region don't affect the original x.

Comparative Insight

Unlike shared, private ensures thread safety but increases memory usage and lacks inter-thread communication.

Task 06

Problem Statement

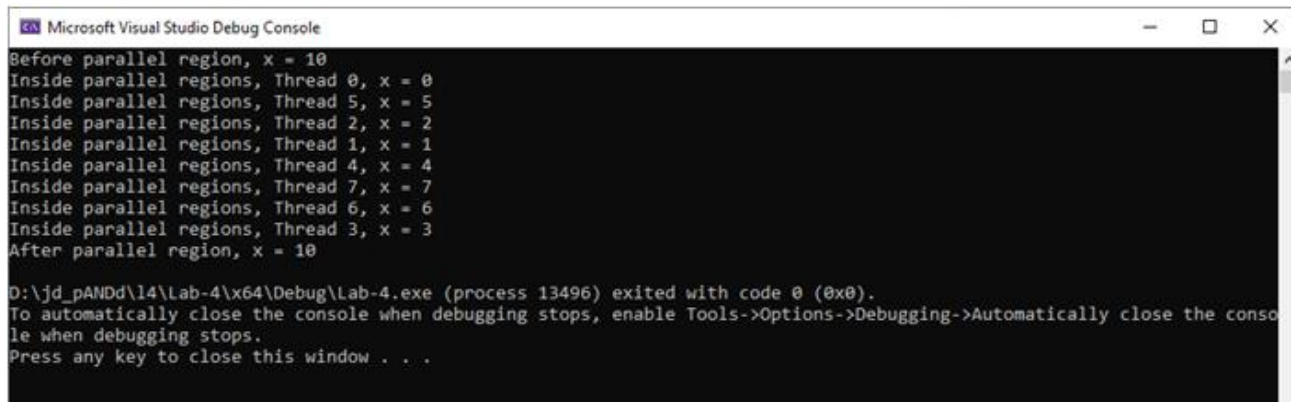
Illustrate how firstprivate initializes private copies with the value of the original variable.

Code

```
#include <iostream>
#include <stdio.h>
#include <omp.h>
int main()
{
```

```
int x = 10;
printf("Before parallel region, x = %d\n", x);
#pragma omp parallel firstprivate(x)
{
    int thread_id = omp_get_thread_num();
    x = thread_id;
    printf("Inside parallel region, Thread %d, x = %d\n", thread_id, x);
}
printf("After parallel region, x = %d\n", x);
return 0;
}
```

Output



```
Microsoft Visual Studio Debug Console
Before parallel region, x = 10
Inside parallel regions, Thread 0, x = 0
Inside parallel regions, Thread 5, x = 5
Inside parallel regions, Thread 2, x = 2
Inside parallel regions, Thread 1, x = 1
Inside parallel regions, Thread 4, x = 4
Inside parallel regions, Thread 7, x = 7
Inside parallel regions, Thread 6, x = 6
Inside parallel regions, Thread 3, x = 3
After parallel region, x = 10

D:\jd_pANDd\l4\Lab-4\x64\Debug\Lab-4.exe (process 13496) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Explanation

- **What:** Similar to private, but initial values are copied from the master variable.
- **Why:** Useful when all threads need the same starting value.
- **How:**
 - Each thread receives a copy of $x = 10$ before execution begins.
 - Modifications inside the region don't affect the main copy.

Comparative Insight

Compared to private, firstprivate adds initialization — preventing undefined variable states.

Task 07

Problem Statement

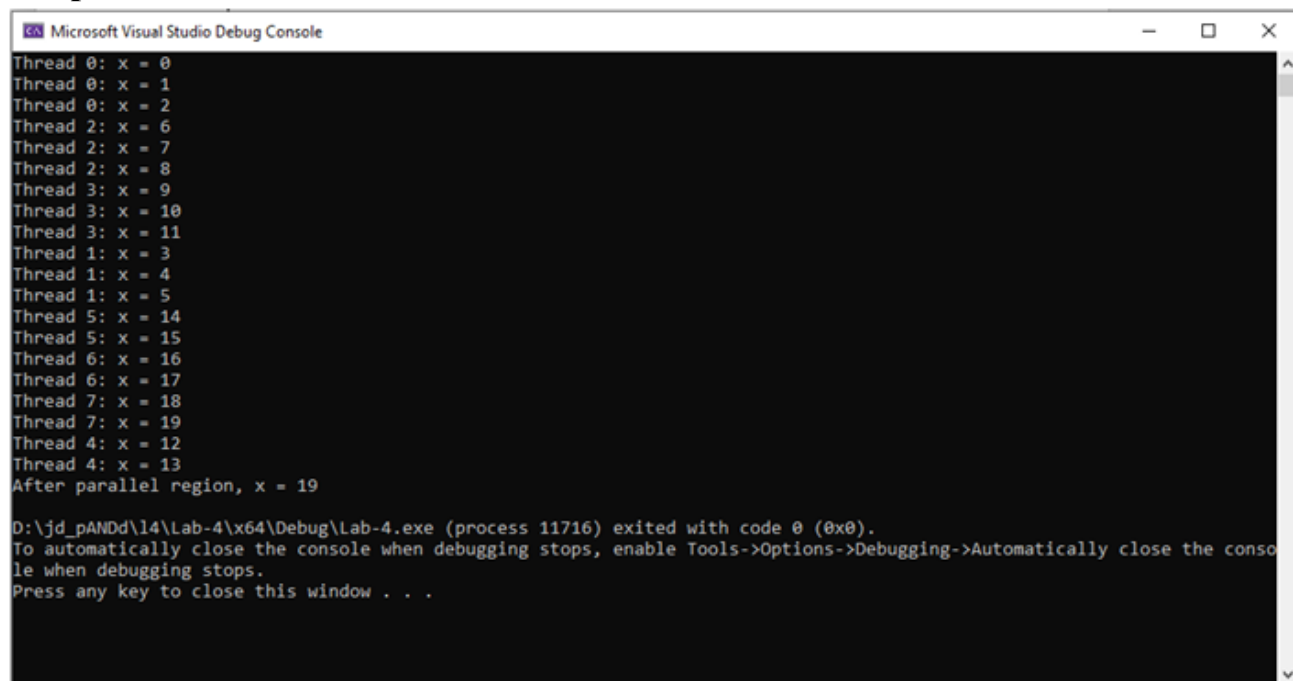
Demonstrate how lastprivate ensures the value from the **last iteration** of a loop is

available outside the parallel region.

Code

```
#include <iostream>
#include <stdio.h>
#include <omp.h>
int main()
{
    int i, x = 0;
#pragma omp parallel for lastprivate(x)
    for (i = 0; i < 20; i++)
    {
        x = i;
    }
    printf("Thread %d: x = %d\n", omp_get_thread_num(), x);
    printf("After parallel region, x = %d\n", x);
    return 0;
}
```

Output



```
Microsoft Visual Studio Debug Console
Thread 0: x = 0
Thread 0: x = 1
Thread 0: x = 2
Thread 2: x = 6
Thread 2: x = 7
Thread 2: x = 8
Thread 3: x = 9
Thread 3: x = 10
Thread 3: x = 11
Thread 1: x = 3
Thread 1: x = 4
Thread 1: x = 5
Thread 5: x = 14
Thread 5: x = 15
Thread 6: x = 16
Thread 6: x = 17
Thread 7: x = 18
Thread 7: x = 19
Thread 4: x = 12
Thread 4: x = 13
After parallel region, x = 19

D:\jd_pANDd\l4\Lab-4\x64\Debug\Lab-4.exe (process 11716) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Explanation

- **What:** Transfers the value of a variable from the last iteration of a loop to the

outer scope.

- **Why:** Ensures consistency of loop-end results after parallel execution.
- **How:**
 - Each thread updates x, but the last iteration's value (highest i) is retained outside.

Comparative Insight

lastprivate bridges parallel and serial scopes — it's essential when post-loop values are required outside.

Overall Comparative Analysis (Across All Tasks)

Feature / Clause	Purpose	Behavior	Advantage	Drawback	Example Tasks
ordered	Maintain output order	Sequential section inside parallel loop	Predictable output	Slower due to synchronization	Task 01
master	Run only by master thread	Exclusive execution by thread 0	Avoids duplicate operations	No implicit barrier	Task 02
shared	Common memory access	All threads access same variable	Data sharing	Race conditions possible	Task 03, 04
private	Each thread has its own copy	Independent execution	Thread safety	No initialization	Task 05
firstprivate	Private with initialization	Copy master value	Safe and initialized	Slight overhead	Task 06
lastprivate	Retain last iteration value	Propagates final value outside loop	Useful for final results	Complex in nested loops	Task 07

Conceptual Relationships

- **private, firstprivate, and lastprivate** are **variable scope modifiers** → control how data behaves across threads.
- **shared** complements them by providing common access.
- **master and ordered** are **execution control clauses** → control *when and by whom* code executes.

- Proper choice of these clauses directly affects **parallel efficiency, synchronization overhead, and correctness.**

Why These Clauses Matter

- They define **data-sharing attributes**, ensuring **thread safety, deterministic results**, and **optimized performance**.
- Overuse of shared causes data races; overuse of private increases memory cost.
- Balance ensures both **speed** and **correctness** in parallel applications.